

Ibadat Singh Khara

Employee id:- 90119849

Q1. Data types for the given columns.

- Ans:** 1) PAN – CHAR(10), because it is exactly 10 characters.
2) Interest amount – NUMERIC(15,2), for exact money values.
3) Number of days – SMALLINT, 3650 fits easily.
4) UPI authorised time – TIMESTAMP.
5) Loan instalment due – DATE.
6) Branch complaint note – TEXT.
7) Transaction counter – BIGINT.
8) Auto-renew – BOOLEAN.

Q2. FLOAT vs NUMERIC for interest amount.

Ans: SQL: SELECT 0.1::FLOAT8 + 0.2::FLOAT8; -- can give 0.30000000000000004
SELECT 0.1::NUMERIC + 0.2::NUMERIC; -- 0.3
For banking amounts, floating-point rounding can cause small money/reconciliation differences.

Q3. Constraint table.

Ans: NOT NULL – value is required. Example: customer name.
UNIQUE – duplicate values are not allowed. Example: receipt_number.
PRIMARY KEY – uniquely identifies each row and cannot be NULL. Example: deposit_id.
FOREIGN KEY – value must exist in another table. Example: customer_id.
CHECK – value must follow a rule. Example: amount >= 500.
DEFAULT – gives a value automatically. Example: status DEFAULT 'ACTIVE'.

Q4. PRIMARY KEY vs UNIQUE.

Ans: A table has one primary key, but can have multiple UNIQUE constraints. A primary key cannot be NULL, while UNIQUE is an alternate uniqueness rule. deposit_id is better as the primary key because it is a dedicated row identifier; receipt_number can stay UNIQUE.

Q5. DELETE, TRUNCATE and DROP.

Ans: DELETE removes rows and allows WHERE. TRUNCATE removes all rows and does not allow WHERE. DROP removes the whole table. DELETE is generally slower for a huge table, while TRUNCATE/DROP are much faster. DELETE triggers can fire; TRUNCATE has separate trigger behavior.

Q6. Seven-year transaction retention.

Ans: Deleting everything older than 90 days is wrong because the bank needs seven years of history. The bank should keep/archive the records for the required seven-year period and only remove them after the retention period.

Q7. NULL queries.

Ans: SELECT NULL = NULL; → NULL
SELECT NULL <> NULL; → NULL
SELECT 5000 + NULL; → NULL
SELECT COALESCE(NULL, NULL, 'FALLBACK'); → FALLBACK
SELECT 'A' || NULL || 'B'; → NULL

Q8. Why the two branch queries have different counts.

Ans: AND has higher priority than OR. The first means branch 1 OR (branch 2 AND amount > 500000). The second requires (branch 1 OR branch 2) AND amount > 500000.

Q9. Quarterly BETWEEN problem.

Ans: If txn_date is a timestamp, DATE '2026-03-31' means the start of March 31, so later transactions that day can be missed. Use:

WHERE txn_date >= TIMESTAMP '2026-01-01 00:00:00' AND txn_date < TIMESTAMP '2026-04-01 00:00:00';

Q10. Why SELECT aliases work in ORDER BY but not WHERE.

Ans: WHERE is evaluated before SELECT, so the alias does not exist yet. ORDER BY happens later, so it can use the SELECT alias.

Q11. Create recurring_deposit.

Ans: CREATE TABLE recurring_deposit (
rd_id BIGINT PRIMARY KEY,
customer_id BIGINT NOT NULL REFERENCES customer(customer_id),
branch_id BIGINT NOT NULL REFERENCES branch(branch_id),
rd_number VARCHAR(20) NOT NULL UNIQUE,
monthly_amount NUMERIC(15,2) CHECK (monthly_amount >= 500),
interest_rate NUMERIC(5,2) CHECK (interest_rate > 0 AND interest_rate <= 12),
tenure_months SMALLINT CHECK (tenure_months BETWEEN 6 AND 120),
start_date DATE NOT NULL DEFAULT CURRENT_DATE,
maturity_date DATE NOT NULL CHECK (maturity_date > start_date),
installments_paid INTEGER NOT NULL DEFAULT 0 CHECK (installments_paid >= 0),
status VARCHAR(10) NOT NULL DEFAULT 'ACTIVE'
CHECK (status IN ('ACTIVE','MATURED','DEFAULTED','CLOSED'))
);

Q12. Six invalid RD inserts.

Ans: 1) monthly_amount = 100 → rejected by monthly amount CHECK.

2) customer_id = 5555 → rejected by customer foreign key.

3) maturity before start → rejected by maturity CHECK.

4) duplicate rd_number → rejected by UNIQUE.

5) status = PAUSED → rejected by status CHECK.

6) installments_paid = -3 → rejected by CHECK.

Example statements can be INSERT statements with those values. Exact error messages depend on the database.

Q13. Insert the three RDs.

Ans: INSERT INTO recurring_deposit

(rd_id,customer_id,branch_id,rd_number,monthly_amount,interest_rate,tenure_months,start_date,maturity_date)
VALUES

(8001,1001,1,'RD2026000001',5000,7.10,24,'2026-03-01','2028-03-01'),
(8002,1005,5,'RD2026000002',2500,6.85,12,'2026-03-05','2027-03-05'),
(8003,1012,4,'RD2026000003',10000,7.35,60,'2026-03-10','2031-03-10');

Q14. Add nominee.

Ans: ALTER TABLE recurring_deposit ADD COLUMN nominee_name VARCHAR(60);

UPDATE recurring_deposit SET nominee_name = 'Meera Reddy' WHERE rd_id = 8001;

Q15. Change minimum monthly instalment to 1000.

Ans: First check: SELECT * FROM recurring_deposit WHERE monthly_amount < 1000;

Then: ALTER TABLE recurring_deposit DROP CONSTRAINT ck_rd_monthly_amount;

ALTER TABLE recurring_deposit ADD CONSTRAINT ck_rd_monthly_amount CHECK (monthly_amount >= 1000);

Q16. Add mandatory scheme_code to populated table.

Ans: Adding VARCHAR(8) NOT NULL directly should fail because old rows have no value. Do it in steps:

ALTER TABLE recurring_deposit ADD COLUMN scheme_code VARCHAR(8);

UPDATE recurring_deposit SET scheme_code = 'RDSAVER' WHERE scheme_code IS NULL;

ALTER TABLE recurring_deposit ALTER COLUMN scheme_code SET NOT NULL;

Q17. Fixed deposits >= 500,000.

Ans: SELECT receipt_number, principal_amount, interest_rate, maturity_date FROM fixed_deposit WHERE
principal_amount >= 500000 ORDER BY principal_amount DESC;

Q18. Deposits booked 1–10 March 2026.

Ans: SELECT * FROM fixed_deposit WHERE booked_date >= DATE '2026-03-01' AND booked_date < DATE '2026-03-11'
ORDER BY booked_date, principal_amount DESC;

Q19. No nominee and nominee not Priya Verma.

Ans: No nominee: SELECT * FROM fixed_deposit WHERE nominee_name IS NULL;

Not Priya, including NULL: SELECT * FROM fixed_deposit WHERE nominee_name <> 'Priya Verma' OR nominee_name IS
NULL;

Simple <> misses NULL because NULL comparisons are UNKNOWN.

Q20. Annual interest, top five.

Ans: SELECT receipt_number, principal_amount, ROUND(principal_amount * interest_rate / 100.0, 2) AS annual_interest
FROM fixed_deposit ORDER BY annual_interest DESC LIMIT 5;

Q21. Customer name and deposit segment.

Ans: SELECT fd.receipt_number,

CONCAT(c.first_name, ' ', c.last_name) AS customer_full_name,

fd.principal_amount,

CASE

WHEN fd.principal_amount >= 1000000 THEN 'HIGH VALUE'

WHEN fd.principal_amount >= 100000 THEN 'STANDARD'

```

ELSE 'SMALL'
END AS segment
FROM fixed_deposit fd
JOIN customer c ON c.customer_id = fd.customer_id
ORDER BY CASE
WHEN fd.principal_amount >= 1000000 THEN 1
WHEN fd.principal_amount >= 100000 THEN 2
ELSE 3 END,
fd.principal_amount DESC;

```

Q22. UPI transactions above 50,000.

Ans: PostgreSQL:

```

SELECT txn_id, account_id, amount, narration FROM account_txn WHERE narration ILIKE '%UPI%' AND amount > 50000
ORDER BY amount DESC LIMIT 10;

```

SQL Server: use SELECT TOP (10) ... ORDER BY amount DESC.

Oracle: use ... ORDER BY amount DESC FETCH FIRST 10 ROWS ONLY.

Q23. Break deposit 7009 and roll back.

Ans: BEGIN;

```

SELECT * FROM fixed_deposit WHERE deposit_id = 7009;

```

```

UPDATE fixed_deposit SET status = 'BROKEN', auto_renew = 'N' WHERE deposit_id = 7009;

```

```

SELECT * FROM fixed_deposit WHERE deposit_id = 7009;

```

```

ROLLBACK;

```

Q24. Mature eligible fixed deposits and roll back.

Ans: BEGIN;

```

SELECT COUNT(*) FROM fixed_deposit WHERE status = 'ACTIVE' AND maturity_date <= DATE '2026-06-30';

```

```

UPDATE fixed_deposit SET status = 'MATURED' WHERE status = 'ACTIVE' AND maturity_date <= DATE '2026-06-30';

```

Check the affected count matches the SELECT count, then ROLLBACK;

Q25. Default and then close RD 8002.

Ans: UPDATE recurring_deposit SET status = 'DEFAULTED' WHERE rd_id = 8002;

```

UPDATE recurring_deposit SET status = 'CLOSED' WHERE rd_id = 8002;

```

Keep installments_paid unchanged. Closing the row instead of deleting it keeps the record/history.

Q26. Vendor sends principal as text.

Ans: Store it as NUMERIC, not VARCHAR. Clean/validate the text in the ingestion/staging layer, convert it to NUMERIC before putting it into the main table, and reject bad values. This avoids parsing the same messy data again and again.

Q27. CASCADE vs RESTRICT for customer deposits.

Ans: Use RESTRICT/NO ACTION. A customer leaving the bank does not mean their old deposits should disappear. CASCADE could delete important financial history automatically. The customer can be marked closed instead. Historical deposits should remain for audit and regulatory purposes.

Q28. Employee authorisation for 40 million transactions.

Ans: Add authorised_by_employee_id, using the same integer type as the employee ID. Make it nullable because it is only required where applicable. Add a foreign key to the employee table. For old rows, add the column first and backfill only when the real authoriser is known; do not invent IDs. Add indexes/constraints carefully because there are 40 million rows.

Q29. CHECK vs lookup table for status.

Ans: CHECK is simple and good when there are only a few fixed values. A lookup table is useful if statuses need extra information or change frequently. For four simple fixed-deposit statuses, I would use the CHECK constraint because it is simpler.

Q30. Production UPDATE with no WHERE and autocommit.

- Ans:**
1. Stop/contain further changes if needed.
 2. Check how many rows were changed and check logs/audit records.
 3. Find a reliable source for the old account statuses, such as backup/WAL/audit data.
 4. Do not blindly change everything back.
 5. Work out the correct old values.
 6. Test the recovery update safely first.
 7. Apply the correction carefully in production.
 8. Check the affected rows and reports afterward.
 9. Document what happened.
 10. Going forward, require a SELECT/row-count check before UPDATE/DELETE and use review/transaction safeguards for production changes.

Done – simple solution format.